Hybrid Genetic Algorithm with Neural Local Search for Large-Scale Traveling Salesman Problems

Jieun Yook
Computer Science and Engineering
Seoul National University
Seoul, South Korea
yookje@snu.ac.kr

***Abstract*—Parallel local ReConstruction (PRC) is a post-processing technique for large-scale TSPs that uses neural networks to iteratively refine partial tours. However, its single-trajectory search leads to diminishing improvements as iterations proceed. We propose EvoPRC, a hybrid genetic algorithm (GA) that integrates PRC as a local search operator. GA enables global exploration through genetic operators, while PRC focuses on local refinement. To enhance PRC, we introduce a descending segment-length schedule that guides the reconstruction process from coarse to fine scales. Experiments show that EvoPRC consistently outperforms PRC-only under the same budget. On TSP-5k, EvoPRC achieves a 1.27 percent gap within 780 iterations, whereas PRC plateaus at 1.43 percent after 1,000 iterations.**

***Index Terms*—Neural Combinatorial Optimization, Hybrid Genetic Algorithm, Memetic Genetic Algorithm**

I. Introduction

The Traveling Salesman Problem (TSP) is a classical combinatorial optimization problem with applications in various domains, including logistics and vehicle routing. As the problem size increases, the search space grows exponentially, highlighting the need for methods that can efficiently generate high-quality approximate solutions. Recently, learning-based Neural Combinatorial Optimization (NCO) frameworks have demonstrated promising performance on large-scale instances [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]. In particular, [15] introduces an approach that combines a lightweight model architecture with iterative post-processing to effectively handle large-scale instances.

In [15], the Parallel Local ReConstruction (PRC) procedure iteratively improves an incumbent tour by partitioning it into contiguous tour segments of size ω. For each segment, PRC fixes the two endpoints and lets the neural network reconstruct the ordering of the internal nodes; if the reconstructed segment yields a lower cost than the original one, it replaces the corresponding segment in the incumbent tour. Multiple non-overlapping segments of size ω are processed in parallel to improve time efficiency, and the initial solution is generated via random insertion [16]. While PRC can effectively refine a solution through partial reconstruction, it inherently operates on a single solution per instance. Consequently, PRC exhibits diminishing returns: as shown in Figure 1, its marginal improvement decreases as the number of iterations increases. To overcome the plateau observed in iterative PRC

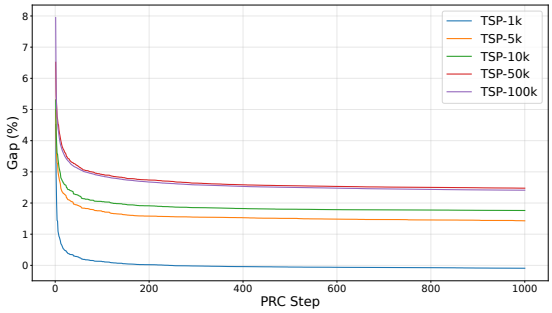


Fig. 1: Optimality gap over PRC iterations starting from a single initial solution for different instance sizes.

on a single solution, we propose a population-based hybrid genetic algorithm [17, 18, 19, 20], EvoPRC, which incorporates PRC as a local search operator. The genetic algorithm (GA) [21, 22] facilitates basin hopping through selection and crossover, while PRC rapidly refines each individual in the population. In this paper, we refer to repeatedly applying PRC to a single solution as PRC-only. Code will be available at https://github.com/yookje/EvoPRC.git.

The contributions of this work are as follows:

- **GA-PRC Hybrid Framework** : We integrate PRC into a GA, extending single-solution refinement to population-based search. This mitigates PRC stagnation, while low-cost population initialization improves scalability to large-scale TSP instances.
- **Descending ω-Scheduling Strategy for PRC** : We introduce a coarse-to-fine schedule for PRC. Since PRC fixes segment endpoints and reorders only internal nodes, using larger segments early provides a broader reconstruction scope, enabling more efficient solution improvement under the same PRC budget.
- **Empirical Validation** : We show that the proposed framework achieves high-quality solutions with fewer PRC calls, whereas a PRC-only fails to reach comparable quality even under a larger PRC budget.

II. Parallel local ReConstruction (PRC)

PRC is an iterative improvement procedure that leverages a trained neural solver θ [23] to refine segments of the

current tour. PRC is applied repeatedly within a predefined computational budget, progressively improving the solution over successive iterations.

At each iteration, PRC selects contiguous tour segments containing ω nodes and treats each segment as an independent subproblem to be solved using the trained neural solver θ. The reconstructed subsequences replace the original segments only when an improved path is found (as in Equation (6)), since the start and end nodes of each segment are fixed, thereby preserving feasibility. Moreover, parallel reconstruction of non-overlapping segments ensures computational efficiency even on large-scale instances. The length ω of the subsequence to reconstruct is sampled from the uniform distribution in Equation (1), following the setting $l_{\max} = 1000$ used in [15].

$$\omega \sim \text{Uniform}[4, l_{\max}] \tag{1}$$

A contiguous tour segment containing ω nodes is formally defined as shown in Eq. (2). The trained solver, θ, sequentially predicts the visiting order of the intermediate nodes, conditioned on the fixed endpoints π_p^1 and π_p^ω, via the conditional probabilities shown in Equation (3), where $\hat{\pi}_p^{2:t-1}$ represents the prefix constructed thus far.

$$\pi_p = (\pi_p^1, \ldots, \pi_p^\omega) \quad p \in \{1, 2, \ldots, \left\lfloor \frac{N}{\omega} \right\rfloor\} \tag{2}$$

$$p_\theta(\hat{\pi}_p^t \mid \pi_p^1, \hat{\pi}_p^{2:t-1}, \pi_p^\omega) \quad t \in \{2, \ldots, \omega - 1\} \tag{3}$$

The trained solver parameterized by θ performs greedy decoding according to Equation (4), where U_t denotes the set of nodes that remain unvisited at time step t. This process yields the reconstructed subsequence as defined in Equation (5), which replaces the original π_p only if the cost condition in Equation (6) is satisfied.

$$\hat{\pi}_p^t = \arg\max_{v \in U_t} p_\theta(v \mid \pi_p^1, \hat{\pi}_p^{2:t-1}, \pi_p^\omega) \tag{4}$$

$$\pi_p' = (\pi_p^1, \hat{\pi}_p^2, \ldots, \hat{\pi}_p^{\omega-1}, \pi_p^\omega) \tag{5}$$

$$cost(\pi_p') \leq cost(\pi_p) \tag{6}$$

For an instance of size N, each PRC iteration reconstructs a total of $\lfloor \frac{N}{\omega} \rfloor$ subproblems in parallel. PRC requires an initial feasible solution in the initial phase, which can be generated from a solution decoded by the trained solver θ, the nearest neighbor heuristic [24, 25], or random insertion [16]. Experimental results in [15] indicate that random insertion provides the best trade-off between performance and computational cost; it is therefore adopted for initialization.

From an operator perspective, PRC can be interpreted as a local search [26] over a neighborhood. However, repeatedly applying PRC to a single solution may converge to a local optimum, stalling further improvement. While the method proposed in [15] mitigates premature convergence to local optima through stochastic mechanisms, including tour-direction reversal, starting-node shifts, and random sampling of ω, its reliance on repeatedly applying PRC to a single solution imposes an inherent limitation. In our framework, we leverage PRC as a learned local improvement operator and integrate it into a population-based search process.

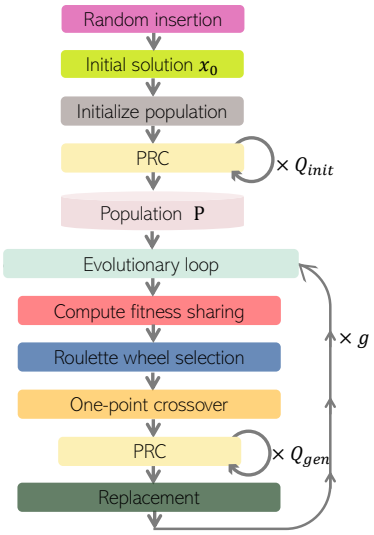


Fig. 2: EvoPRC framework.

Algorithm 1: Hybrid GA with PRC refinement

Input: population size r, PRC iterations Q_{init}, Q_{gen}, generations g
Output: best solution $x^\star$
$x_0 \leftarrow$ RANDOMINSERTION()
$\mathcal{P} \leftarrow$ INITPOPULATION(x_0, r)
$\mathcal{P} \leftarrow$ PRC$(\mathcal{P}, Q_{\text{init}})$
for $t \leftarrow 1$ **to** g **do**
  $\tilde{f} \leftarrow$ COMPUTEFITNESSSHARING$(\mathcal{P})$ // see Algorithm 3
  $(parent_1, parent_2) \leftarrow$ ROULETTEWHEELSELECT$(\mathcal{P}, \tilde{f})$
  $offspring \leftarrow$ ONEPOINTCROSSOVER$(parent_1, parent_2)$
  $offspring \leftarrow$ PRC$(offspring, Q_{\text{gen}})$
  $\mathcal{P} \leftarrow$ REPLACE$(\mathcal{P} \cup \{offspring\}, r)$ // keep the best r solutions
return $x^\star$

III. METHODOLOGY

A. EvoPRC Framework

This study focuses on the symmetric TSP. Algorithm 1 and Figure 2 provide an overview of the proposed EvoPRC framework. The algorithm takes as inputs the population size r, the number of PRC iterations used during the initialization phase Q_{init}, the number of PRC iterations applied at each generation Q_{gen}, and the maximum number of generations g. Fitness sharing [27] guides roulette wheel parent selection [28], followed by one-point crossover [21, 29]. The offspring is evaluated together with the current population, and only the

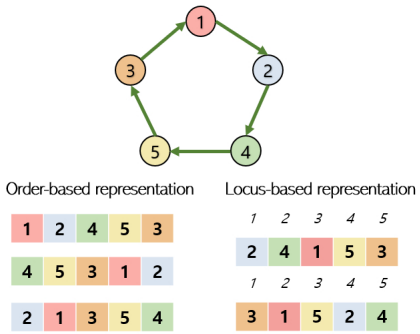


Fig. 3: An example of order-based and locus-based representations for a graph, where different genotypes correspond to the same phenotype.

top r solutions are retained for the next generation. EvoPRC returns the best solution $x^\star$ found during the search, and instance-dependent parameter settings are reported in Table II.

B. Solution Representation and Initialization

1) Order-Based and Locus-Based Representations: Order-based representations [30] explicitly encode the visiting order of cities, and crossover operators are applied directly to these permutations. However, because a TSP tour is cyclic, the same tour can be represented by multiple genotypes depending on the choice of starting city and traversal direction (Figure 3).

It may lead to a distorted view of population diversity, as order-based similarity measures overlook representational equivalence.

Locus-based representation [31, 32] encodes the connectivity between cities, where in the successor-based form, the value at index i indicates the city visited immediately after city i, and in the predecessor-based form, it indicates the city visited immediately before city i (Figure 3). To quantify population diversity, we measure inter-individual similarity using the Hamming distance. Given that a locus-based representation admits two forms—successor and predecessor—we first select the representation with the smaller first-visited city index to obtain a standardized locus representation, and then compute the Hamming distance [33, 34] accordingly (Algorithm 2).

2) Cost-Efficient Population Initialization: As shown in Table I, random insertion exhibits a lower computational cost than greedy decoding ($\omega = N$); However, the larger the instance size, the longer the time required to construct an initial tour. Specifically, the time to generate an initial solution via random insertion is approximately 0.01 minutes for TSP-1k, while it takes 4.08 minutes for TSP-50k and 20.63 minutes for TSP-100k.

Generating r initial solutions via random insertion would increase the initialization cost by a factor of r. To reduce this overhead, we initialize the population by deriving r distinct order-based representations from an initial solution x_0 through variations such as changing the starting point or reversing the tour direction. This strategy maintains computational efficiency while allowing distinct order-based representations

Algorithm 2: COMPUTEHAMMINGDISTANCE

Input: Order-based population $\Pi = \{\pi_1, \ldots, \pi_r\}$, tour size N
Output: Pairwise Hamming distance matrix $D \in \mathbb{R}^{r \times r}$

```
// Step 1: convert to locus-based (successor) represention
```
for $i \leftarrow 1$ **to** r **do**
  $l_i \leftarrow$ ORDERTOLOCUS(π_i)

```
// Step 2: construct predecessor representation and choose the one with the smaller first-visited city index
```
for $i \leftarrow 1$ **to** r **do**
  $l_i \leftarrow$ STANDARDIZELOCUS(l_i)

```
// Step 3: compute pairwise Hamming distances in locus space
```
for $i \leftarrow 1$ **to** r **do**
  for $j \leftarrow 1$ **to** r **do**
    $D_{ij} \leftarrow \sum_{n=1}^{N} \mathbb{I}[l_i(n) \neq l_j(n)]$

return D

TABLE I: Comparison of runtime and solution quality across population-initialization methods. Results are reported for instance sizes of 1k, 5k, 10k, 50k, and 100k, with 128, 16, 16, 16, and 16 instances, respectively.

	TSP-1k		TSP-5k		TSP-10k		TSP-50k		TSP-100k	
Method	Gap	Time	Gap	Time	Gap	Time	Gap	Time	Gap	Time
Random insertion	12.33	0.01m	13.78	0.02m	14.89	0.09m	14.17	4.08m	14.21	20.63m
Greedy	1.48	0.21m	3.17	0.81m	3.64	2.92m	5.32	1.18h	6.19	5.30h

of the same phenotype to generate diverse offspring through crossover. Consequently, it facilitates transitions to diverse basins of attraction associated with different local optima during the search.

C. Genetic Operator Design

1) Fitness Sharing: With a small population size, insufficient diversity can lead to premature convergence. To mitigate this issue, we apply fitness sharing during parent selection (Algorithm 3), where roulette wheel selection employs fitness sharing [27] values $\tilde{f}_i$.

Each individual's tour length is first normalized using min–max scaling to obtain the raw fitness f_i. Fitness sharing $\tilde{f}_i$ is then defined as

$$\tilde{f}_i = \frac{f_i}{m_i}, \tag{7}$$

where m_i denotes the niche count of individual i, capturing its similarity to other individuals in the population. The niche count is computed as

$$m_i = \sum_{j=1}^{r} sh(D_{ij}) + \epsilon, \tag{8}$$

where $\epsilon \ll 1$ is a small constant introduced to prevent division by zero. The function $sh(\cdot)$ is a sharing function defined over

Algorithm 3: COMPUTEFITNESSSHARING

Data: tour lengths $L_1, \ldots, L_r$, distance matrix D, niche radius σ, decay α, small constant ϵ
Result: fitness sharing $\tilde{f}_1, \ldots, \tilde{f}_r$

```
for i ← 1 to r do
    m_i ← 0
    for j ← 1 to r do
        if D_ij < σ then
            sh ← 1 − (D_ij / σ)^α
        else
            sh ← 0
        m_i ← m_i + sh
    m_i ← max(m_i, ε)   // avoid division by zero
L_min ← min_i L_i,   L_max ← max_i L_i
for i ← 1 to r do
    L̂_i ← MINMAXNORMALIZATION(L_i, L_min, L_max)
    f_i ← 1 − L̂_i                 // larger is better
    f̃_i ← f_i / m_i
return {f̃_i}_{i=1}^r
```

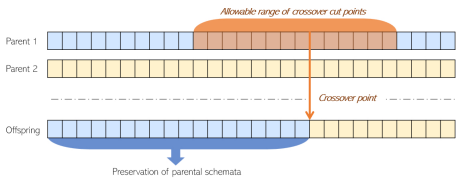


Fig. 4: TSP crossover operator preserving parental schemata through constrained cut-point ranges.

the distance d between individuals, parameterized by the niche radius σ and the decay coefficient α:

$$sh(d) = \begin{cases} 1 - \left(\dfrac{d}{\sigma}\right)^{\alpha}, & \text{if } d < \sigma, \\ 0, & \text{if } d \geq \sigma. \end{cases} \tag{9}$$

The niche radius σ determines the range within which individuals are regarded as similar neighbors. Larger values of the parameter α impose a stronger penalty on similar individuals. The distance D_{ij} denotes the Hamming distance between individuals i and j, computed using the standardized locus representation (Section III-B1).

2) One-point Crossover: Offspring are generated via one-point crossover, where the cut point is constrained to preserve parental schemata. The cut point c is selected from the range $c \in [N - \lambda_{\text{tail}},\ N - 4]$. If an infeasible solution arises after crossover, solution repair is applied only to the segment following the cut point. As a result, the offspring inherits schemata of length at least $N - \lambda_{\text{tail}}$ throughout the crossover process, effectively preventing the disruption of high-quality partial tour structures (Figure 4).

3) PRC as a Local Search Operator: PRC is applied in two stages (Figure 2). During the initialization phase, PRC is applied Q_{init} times to every individual in the population to refine the initial solutions. As a result, the evolutionary process is initialized with high-quality solutions. In each generation, PRC is applied Q_{gen} times to a offspring produced by crossover. While crossover promotes exploration by combining parental structures and moving the search into new basins of attraction, PRC refines the resulting solutions within those basins toward local optima. By coupling these complementary roles, EvoPRC mitigates the tendency of PRC-only search to become trapped in suboptimal basins.

As described in Section II, PRC reconstructs a partial tour of length ω while fixing the start and end nodes, where a set of ω values is sampled once in advance according to Equation (1), with the total number of samples equal to the PRC budget. If high-cost partial tours of large cardinality are not covered by the ω-length segments, improvements in the overall tour cost may saturate. To mitigate this limitation, we introduce a descending ω-Scheduling Strategy. Specifically, the ω values sampled from Equation (1) are sorted in descending order and then applied sequentially in PRC. In the early stages, larger ω values are used to improve high-cost partial tours containing more nodes, thereby modifying larger portions of the tour. The schedule then gradually shifts to smaller ω values for fine-grained refinement. This coarse-to-fine strategy first adjusts the global tour structure and subsequently enables local optimization around the improved structure, leading to more efficient tour improvement under the same PRC budget. In Section IV-G, we experimentally show that the descending schedule outperforms the random and ascending schedules.

IV. EXPERIMENTS

A. Datasets

We evaluate our method on the synthetic TSP instances introduced in [15]. Each instance consists of vertices independently sampled from a uniform distribution over the two-dimensional Euclidean space $[0, 1] \times [0, 1]$. The ground-truth solutions are computed using LKH3 [35]. For evaluation, we use 128 instances for TSP-1k, and 16 instances each for TSP-5k, TSP-10k, TSP-50k, and TSP-100k.

B. Evaluation Metrics

These experiments employed the optimality gap as the evaluation metric, defined as follows:

$$\text{Gap}(\%) = \frac{L_{\text{pred}} - L_{\text{ground}}}{L_{\text{ground}}} \times 100 \tag{10}$$

where L_{pred} denotes the tour length obtained by the method, and L_{ground} denotes the ground-truth tour length generated by LKH3 [35]. Lower gap values indicate better performance. Since LKH3 solutions are not guaranteed to be globally optimal, negative gaps may occur when the evaluated method finds shorter tours than the ground truth.

C. Implementation Details

All experiments were conducted on the following hardware configuration: two AMD EPYC™ 9354 Processors (32 cores/64 threads, 3.25 GHz) for CPU and a single NVIDIA

TABLE II: Our EvoPRC parameters are chosen based on TSP instance size N. λ_{tail} guarantees preservation of at least $N - \lambda_{tail}$ schemata.

Parameter	N				
	1k	**5k**	**10k**	**50k**	**100k**
Population size	4	4	4	4	4
Max generations	10	10	10	8	5
α_{shared}	2	2	2	2	2
σ_{shared}	200	1,000	2,000	10,000	20,000
Selection pressure	4	4	4	4	4
Offspring	1	1	1	1	1
λ_{tail}	10	50	50	50	50
Q_{init}	20	20	20	30	50
Q_{gen}	70	70	70	100	150
$Budget_{total}$	780	780	780	920	950

H100 NVL 94GB PCIe GPU. To ensure reproducibility of the results reported in Table III and Table V, all experiments used a fixed random seed[1]. Batch sizes are the same as those used in [15]: 128 for TSP-1k, 16 for TSP-5k and TSP-10k, 8 for TSP-50k, and 4 for TSP-100k. Table II presents the EvoPRC parameter settings across instance sizes. Population size r was fixed at 4 for all instances, balancing genetic diversity with maintenance costs in large-scale TSP. The niching radius for fitness sharing, σ_{shared}, was set to $0.2N$ proportional to instance size N, with decay coefficient $\alpha_{\text{shared}} = 2$.

D. Evaluation Protocol : PRC Budget

To evaluate the effectiveness of the proposed EvoPRC framework, we compare it against several baseline methods. All PRC-based baselines listed in Tables III and Table V are initialized from the same random insertion tours reported in Table I, ensuring a consistent experimental setup.

a) ***PRC-only****:* This baseline applies the PRC operator repeatedly to a single initial solution. A total of $Budget_{total}$ PRC iterations is used to define the evaluation protocol across all experiments.

b) ***EvoPRC****:* The proposed hybrid GA integrates PRC as a local search operator within genetic algorithm framework. In this setting, the PRC budget accumulates over generations as the algorithm proceeds. $Generation_0$ corresponds to the initialization phase, during which each individual in the population of size r undergoes Q_{init} PRC iterations, which can be formally expressed as follows.

$$\text{Budget}_{\text{evo}}^{(0)} = r \times Q_{\text{init}} \tag{11}$$

In $Generation_i (i > 0)$, a single offspring is produced in each iteration, and it is fine-tuned by applying Q_{gen} PRC operations. Consequently, the cumulative PRC budget across generations can be computed as follows.

$$\text{Budget}_{\text{evo}}^{(i)} = r \times Q_{\text{init}} + i \times Q_{\text{gen}}, \quad \text{where } i > 0 \tag{12}$$

[1]Implemented with `torch.cuda.manual_seed(123)`

c) ***Multi-Trajectory PRC****:* Similar to EvoPRC, it starts from a tour constructed using random insertion, generates r augmented solutions, and iteratively applies PRC to each solution. Unlike EvoPRC, however, it does not employ genetic operators. This baseline is designed to evaluate the effect of repeatedly applying PRC across multiple trajectories without evolutionary operations. When PRC is applied Q_{multi} times to each solution, the total PRC budget is given by

$$\text{Budget}_{\text{multi}} = r \times Q_{\text{multi}}. \tag{13}$$

d) ***PRC-Solver Sampling****:* It uses the same trained neural solver θ as PRC, but generates a complete tour through a single greedy decoding pass with $\omega = N$, rather than reconstructing partial tours. Decoding is performed from 20 distinct starting nodes, and the lowest optimality gap among the resulting tours is reported. This baseline evaluates the standalone solution quality of the neural solver without any iterative refinement.

Under the evaluation protocol defined by $Budget_{\text{total}}$, the following budget equivalence holds:

$$\text{Budget}_{\text{total}} = \text{Budget}_{\text{evo}}^{g} = \text{Budget}_{\text{multi}} \tag{14}$$

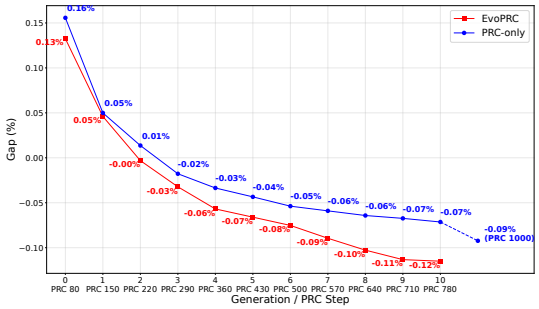
E. Experimental Results

Table III compares our method with baseline approaches across different instance sizes. The total budget for each instance follows the settings in Table II, and PRC budgets for all baselines are allocated according to the protocol in Section IV-D. The EvoPRC framework consistently achieves the lowest optimality gaps across all instance sizes. In particular, on TSP-5k instances, EvoPRC attains an optimality gap of 1.27%, significantly outperforming PRC-only (1.46%) and Multi-Trajectory PRC (1.45%). Despite using multiple initial solutions, Multi-Trajectory PRC exhibits consistently inferior performance compared to EvoPRC. This suggests that EvoPRC more effectively leverages the strengths of genetic algorithms in exploring large-scale search spaces.

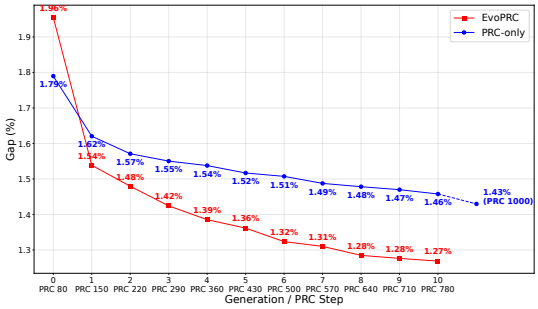
TABLE III: Comparison of results on synthetic TSP instances.

Method \ Instance size	1k Gap	5k Gap	10k Gap	50k Gap	100k Gap
PRC-only	-0.09	1.46	1.78	2.49	2.41
Multi-Trajectory PRC	-0.11	1.45	1.90	2.67	2.61
EvoPRC	-0.12	1.27	1.72	2.43	2.37

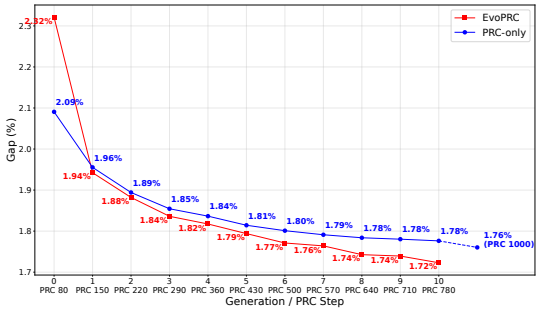
Figure 5 compares the generation-wise best-so-far optimality gaps of PRC-only and EvoPRC across different instance sizes. The x-axis jointly displays the generation index and the cumulative PRC budget, enabling a direct comparison under the same computational cost. From $Generation_1$ onward, EvoPRC maintains lower optimality gaps than PRC-only and yields better solutions with a smaller budget before reaching $Budget_{total} = 1000$ across all instance sizes. These results indicate that combining genetic operators with PRC can provide a more efficient search than single-trajectory iterative refinement.



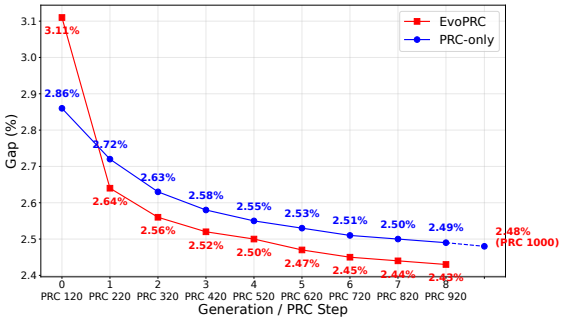
(a) TSP-1k



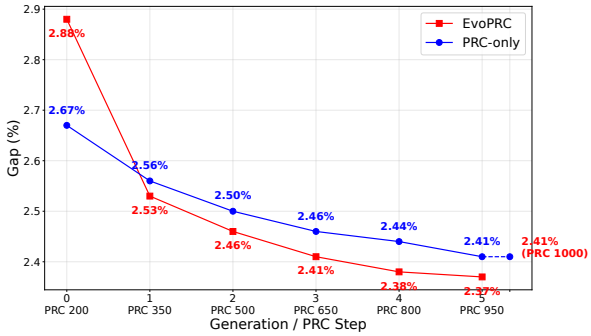
(b) TSP-5k



(c) TSP-10k



(d) TSP-50k



(e) TSP-100k

Fig. 5: Generation-wise best-so-far optimality gap of PRC-only and EvoPRC under equivalent PRC budgets.

F. Computational Efficiency Analysis

Table IV reports the runtime and memory usage averaged over five independent runs on four TSP-5k instances. Runtime is measured in minutes and memory usage in MB. PRC-Solver Sampling performs inference with $\omega = N$ from 20 different starting nodes, while the total budget is fixed to $\text{Budget}_{\text{total}} = \text{Budget}^3_{\text{evo}} = 290$ for all methods.

PRC-Solver Sampling relies on start-point diversification, with PRC refinement applied only once to the entire tour, yielding relatively larger optimality gaps. Moreover, parallel greedy decoding from multiple starting nodes leads to the highest memory consumption among the compared methods.

Although EvoPRC achieves the smallest optimality gap, it requires approximately 2.8 times as much memory as PRC-only because of its population-based processing. Despite this higher memory usage, EvoPRC exhibits a shorter runtime than PRC-only, as PRC-only performs 290 PRC iterations sequentially on a single solution, whereas EvoPRC executes $r \times Q_{\text{init}}$ PRC operations in parallel.

TABLE IV: Comparison of memory usage and inference time across methods. Results obtained from five independent runs on four TSP-5k datasets.

Method	Best	Worst	Avg (std)	Time	Memory
PRC-Solver Sampling	1.86	2.19	2.06 (0.11)	14.23	3387.13
PRC-only	1.45	1.64	1.53 (0.07)	14.49	**458.945**
EvoPRC	**1.36**	**1.47**	**1.45 (0.04)**	**11.39**	1273.49

G. Design Analysis

The following experiments were conducted on a set of 16 TSP-5k instances.

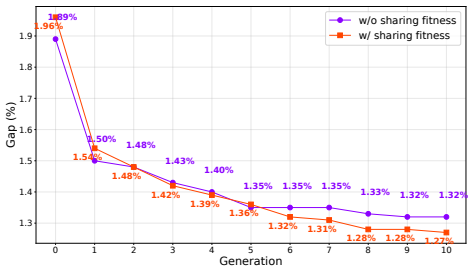
a) ***Fitness Sharing Effect****:* Table V and Figure 6 compare the performance of EvoPRC under fitness sharing and

length-only fitness settings. With fitness sharing, the optimality gap of EvoPRC converges to a lower value as generations progress, and the final gap is also smaller than that obtained with length-only fitness. These results suggest that fitness sharing helps maintain population diversity, thereby mitigating premature convergence.

TABLE V: Comparison of fitness sharing and length-only fitness in EvoPRC.

Configuration	Gap
Length-only fitness	1.32
Fitness sharing	**1.27**

Fig. 6: Generation-wise best-so-far optimality gap of EvoPRC with fitness sharing and length-only fitness on TSP-5k.



b) ***Comparison of ω-Scheduling Strategies****:* Table VI presents the performance of three ω scheduling strategies: random, ascending, and descending. The results are obtained from five independent runs of the PRC-only model with a total PRC budget of 70. On average, the descending schedule reduces the optimality gap by 0.13 and 0.25 percentage points compared with the random and ascending schedules, respectively, demonstrating the effectiveness of coarse-to-fine optimization. Since PRC rearranges only the internal nodes of a segment while fixing its endpoints, using a larger ω in the early stage to improve the global structure of the tour, followed by smaller ω values for fine-grained refinement, provides a more efficient optimization strategy.

TABLE VI: Performance differences across ω-Scheduling Strategy were evaluated on four TSP-5k datasets.

Configuration	Best	Worst	Avg (std)
Random	1.20	1.22	1.22 (0.01)
Ascending order	1.34	1.34	1.34 (0.00)
Descending order	**1.07**	**1.16**	**1.09 (0.03)**

V. Conclusion

This study proposes EvoPRC, a hybrid genetic algorithm that integrates PRC as a local search operator. Although PRC refines a solution using a neural network solver, its single-trajectory nature leads to diminishing returns. By integrating the global exploration of GA with the local refinement of PRC, EvoPRC overcomes this limitation. Experiments show that EvoPRC consistently outperforms PRC-only and Multi-Trajectory PRC under equivalent budgets. On TSP-5k, EvoPRC achieves a 1.27% gap with 780 PRC calls, while PRC-only plateaus at 1.43% after 1,000 calls. Fitness sharing and descending ω-scheduling further improve performance. Hyperparameter sensitivity analysis and extension to other combinatorial problems such as vehicle routing remain as future work.

Acknowledgment

The ICT at Seoul National University provides research facilities for this study. This research was supported by the Challengeable Future Defense Technology Research and Development Program through the Agency for Defense Development (ADD), funded by the Defense Acquisition Program Administration (DAPA) in 2025 (No. 915110201).

References

[1] F. Luo, X. Lin, F. Liu, Q. Zhang, and Z. Wang, "Neural combinatorial optimization with heavy decoder: Toward large scale generalization," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[2] H. Fang, Z. Song, P. Weng, and Y. Ban, "INViT: A generalizable routing problem solver with invariant nested view transformer," in *International Conference on Machine Learning*, 2024.

[3] D. Drakulic, S. Michel, F. Mai, A. Sors, and J.-M. Andreoli, "BQ-NCO: Bisimulation quotienting for efficient neural combinatorial optimization," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[4] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min, "POMO: Policy optimization with multiple optima for reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 21 188–21 198.

[5] Y. Jin, Y. Ding, X. Pan, K. He, L. Zhao, T. Qin, L. Song, and J. Bian, "Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 7, 2023, pp. 8132–8140.

[6] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://openreview.net/forum?id=ByxBFsRqYm

[7] Z. Sun and Y. Yang, "Difusco: Graph-based diffusion solvers for combinatorial optimization," *Advances in Neural Information Processing Systems*, vol. 36, pp. 3706–3731, 2023.

[8] Y. Li, J. Guo, R. Wang, H. Zha, and J. Yan, "Fast t2t: Optimization consistency speeds up diffusion-based training-to-testing solving for combinatorial optimization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 30 179–30 206, 2024.

[9] M. Kim, J. Park, and J. Park, "Sym-nco: Leveraging symmetricity for neural combinatorial optimization,"

Advances in Neural Information Processing Systems, vol. 35, pp. 1936–1949, 2022.

[10] Y. Ma, J. Li, Z. Cao, W. Song, L. Zhang, Z. Chen, and J. Tang, "Learning to iteratively solve routing problems with dual-aspect collaborative transformer," *Advances in Neural Information Processing Systems*, vol. 34, pp. 11 096–11 107, 2021.

[11] Y. Xiao, D. Wang, R. Cao, X. Wu, B. Li, and Y. Zhou, "Geld: A unified neural model for efficiently solving traveling salesman problems across different scales," *arXiv preprint arXiv:2506.06634*, 2025.

[12] X. Pan, Y. Jin, Y. Ding, M. Feng, L. Zhao, L. Song, and J. Bian, "H-tsp: Hierarchically solving the large-scale traveling salesman problem," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 8, 2023, pp. 9345–9353.

[13] H. Ye, J. Wang, H. Liang, Z. Cao, Y. Li, and F. Li, "Glop: Learning global partition and local construction for solving large-scale routing problems in real-time," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 18, 2024, pp. 20 284–20 292.

[14] Y. Li, J. Guo, R. Wang, and J. Yan, "T2t: From distribution learning in training to gradient search in testing for combinatorial optimization," *Advances in Neural Information Processing Systems*, vol. 36, pp. 50 020–50 040, 2023.

[15] Z. Wang, F. Luo, J. Qiu, F. Liu, and Q. Zhang, "Boosting neural combinatorial optimization for large-scale vehicle routing problems," in *International Conference on Learning Representations*, 2025.

[16] D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

[17] P. Moscato, "On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms," California Institute of Technology, Tech. Rep. Caltech Concurrent Computation Program Report 826, 1989.

[18] S. Jung and B. R. Moon, "A hybrid genetic algorithm for the vehicle routing problem with time windows." in *GECCO*, 2002, pp. 1309–1316.

[19] R. Kumar and M. Memoria, "A review of memetic algorithm and its application in traveling salesman problem," *Int. J. Emerg. Technol*, vol. 11, no. 2, pp. 1110–1115, 2020.

[20] A. T. Al-Taani and L. M. Al-Afifi, "Solving the multiple traveling salesman problem using memetic algorithm," *Artificial Intelligence Evolution*, pp. 27–40, 2022.

[21] J. H. Holland, *Adaptation in Natural and Artificial Systems*. MIT Press, 1992, first published 1975.

[22] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[24] E. L. Lawler, J. K. Lenstra, A. H. G. R. Kan, and D. B. Shmoys, Eds., *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. John Wiley & Sons, 1985.

[25] D. J. Rosenkrantz, R. E. Stearns, and I. Philip M. Lewis, "An analysis of several heuristics for the traveling salesman problem," *SIAM Journal on Computing*, vol. 6, no. 3, pp. 563–581, 1977.

[26] C. Rego, D. Gamboa, F. Glover, and M. Laguna, "Traveling salesman problem heuristics: Leading methods, implementations and latest advances," *European Journal of Operational Research*, vol. 211, no. 3, pp. 427–441, 2011.

[27] D. E. Goldberg and J. Richardson, "Genetic algorithms with sharing for multimodal function optimization," in *Proceedings of the Second International Conference on Genetic Algorithms*. Lawrence Erlbaum Associates, 1987, pp. 41–49.

[28] D. E. Goldberg and K. Deb, "A comparative analysis of selection schemes used in genetic algorithms," in *Foundations of genetic algorithms*. Elsevier, 1991, vol. 1, pp. 69–93.

[29] H.-S. Yoon and B.-R. Moon, "An empirical study on the synergy of multiple crossover operators," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 212–223, 2002.

[30] J. Grefenstette, R. Gopal, B. Rosmaita, and D. Van Gucht, "Genetic algorithms for the traveling salesman problem," in *Proceedings of the first International Conference on Genetic Algorithms and their Applications*. Psychology Press, 2014, pp. 160–168.

[31] P. Larranaga, C. M. H. Kuijpers, R. H. Murga, I. Inza, and S. Dizdarevic, "Genetic algorithms for the travelling salesman problem: A review of representations and operators," *Artificial intelligence review*, vol. 13, no. 2, pp. 129–170, 1999.

[32] T. N. Bui and B. R. Moon, "A new genetic approach for the traveling salesman problem," in *Proceedings of the First IEEE Conference on Evolutionary Computation. IEEE World Congress on Computational Intelligence*. IEEE, 1994, pp. 7–12.

[33] R. W. Hamming, "Error detecting and error correcting codes," *The Bell system technical journal*, vol. 29, no. 2, pp. 147–160, 1950.

[34] F.-I. Chou, W.-H. Ho, and C.-H. Chen, "Niche genetic algorithm for solving multiplicity problems in genetic association studies." *Intelligent Automation & Soft Computing*, vol. 26, no. 3, 2020.

[35] K. Helsgaun, "An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems," *Roskilde: Roskilde University*, vol. 12, pp. 966–980, 2017.